

# CS23232-Fundamentals of Data Structures Using C-2025 Batch

|              |                              |
|--------------|------------------------------|
| Started on   | Friday, 15 May 2026, 6:58 PM |
| State        | Finished                     |
| Completed on | Friday, 15 May 2026, 7:16 PM |
| Time taken   | 18 mins 19 secs              |
| Name         | BHUVANA S 2025-BME           |

**Question 1**

Correct

Marked out of  
100Flag  
question

## Problem Statement:

Write a program to implement **Binary Search** to find a given element in a **sorted array** of integers.

The program should repeatedly divide the search interval into halves and determine whether the element is present or not.

## Important Note:

- The input array **must be sorted in ascending order**.
- Location is based on **0-based indexing** (as reflected in the sample output).

## Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (sorted array elements).
3. An integer **key** to be searched.

## Output Format:

- If the element is found, print:  
`Element found at location : X`
- If the element is not found, print:  
`Element not found.`

Sample Input 1:

```
5
10 20 30 40 50
30
```

Sample Output 1:

```
Element found at location : 2
```

Sample Input 2:

```
5
10 20 30 40 50
55
```

Sample Output 2:

```
Element not found.
```

Sample Output 2:

Element not found.

For example:

| Input                     | Result                        |
|---------------------------|-------------------------------|
| 5<br>10 20 30 40 50<br>30 | Element found at location : 2 |
| 5<br>10 20 30 40 50<br>55 | Element not found.            |

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main(){
3     int n,key,i;
4     if(scanf("%d", &n)!=1)
5         return 0;
6     int arr[n];
7     for(i=0;i<n;i++){
8         scanf("%d", &arr[i]);
9     }
10
11     scanf("%d", &key);
12     int low=0;
13     int high=n-1;
14     int mid;
15     int foundIndex=-1;
16     while(low<=high){
17         mid=low+(high-low)/2;
18         if(arr[mid]==key){
19             foundIndex=mid;
20             break;
21         }
22         else if(arr[mid]<key){
```

|   | Input                     | Expected                      | Got                           |   |
|---|---------------------------|-------------------------------|-------------------------------|---|
| ✓ | 5<br>10 20 30 40 50<br>30 | Element found at location : 2 | Element found at location : 2 | ✓ |
| ✓ | 5<br>10 20 30 40 50<br>55 | Element not found.            | Element not found.            | ✓ |

Passed all tests! ✓

Finish review

Quiz navigation

1

Finish review

|                     |                              |
|---------------------|------------------------------|
| <b>Started on</b>   | Friday, 15 May 2026, 6:49 PM |
| <b>State</b>        | Finished                     |
| <b>Completed on</b> | Friday, 15 May 2026, 6:58 PM |
| <b>Time taken</b>   | 8 mins 16 secs               |
| <b>Name</b>         | BHUVANA S 2025-BME           |

Question **1**

Correct

Marked out of 100

Flag question

Write a program to implement **Linear Search** to find a given element in a list of integers.

The program should traverse the array sequentially and determine whether the element is present or not.

## Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).
3. An integer **key** to be searched.

## Output Format:

- If the element is found, print:  
Element found at location : X  
(Location starts from 1)
- If the element is not found, print:  
Element not found.

Sample Input 1:

5  
30 40 50 20 10  
20

Sample Output 1:

Element found at location : 3

Sample Input 2:

5  
30 40 50 20 10  
55

Sample Output 2:

Element not found.

For example:

| Input                     | Result                        |
|---------------------------|-------------------------------|
| 5<br>30 40 50 20 10<br>20 | Element found at location : 3 |
| 5<br>30 40 50 20 10<br>55 | Element not found.            |

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main(){
3     int n,key,i;
4     int found=0;
5     if(scanf("%d", &n)!=1)
6         return 0;
7     int arr[n];
8     for(i=0;i<n;i++){
9         scanf("%d", &arr[i]);
10    }
11    scanf("%d", &key);
12    for(i=0;i<n;i++){
13        if(arr[i]==key){
14            printf("Element found at location : %d\n",i);
15            found=1;
16            break;
17        }
18    }
19    if(!found){
20        printf("Element not found.\n");
21    }
22    return 0;

```

|   | Input                     | Expected                      | Got                           |   |
|---|---------------------------|-------------------------------|-------------------------------|---|
| ✓ | 5<br>30 40 50 20 10<br>20 | Element found at location : 3 | Element found at location : 3 | ✓ |
| ✓ | 5<br>30 40 50 20 10<br>55 | Element not found.            | Element not found.            | ✓ |

# CS23232-Fundamentals of Data Structures Using C-2025 Batch

|                     |                              |
|---------------------|------------------------------|
| <b>Started on</b>   | Friday, 15 May 2026, 7:17 PM |
| <b>State</b>        | Finished                     |
| <b>Completed on</b> | Friday, 15 May 2026, 7:28 PM |
| <b>Time taken</b>   | 11 mins 4 secs               |
| <b>Name</b>         | BHUVANA S 2025-BME           |

## Question 1

Correct

Marked out of 100

Flag question

## Problem Statement:

Write a program to sort a list of integers in **ascending order** using the **Insertion Sort** algorithm.

Insertion Sort works by taking one element at a time and inserting it into its correct position in the already sorted part of the array.

## Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

## Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5  
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

| Input               | Result         |
|---------------------|----------------|
| 5<br>30 40 50 20 10 | 10 20 30 40 50 |

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 void insertionSort(int arr[],int n){
3     int i,key,j;
4     for(i=1;i<n;i++){
5         key=arr[i];
6         j=i-1;
7         while(j>=0&&arr[j]>key){
8             arr[j+1]=arr[j];
9             j=j-1;
10        }
11        arr[j+1]=key;
12    }
13 }
14 int main(){
15     int n;
16     if(scanf("%d", &n) !=1)
17         return 0;
18     int arr[n];
19     for(int i=0;i<n;i++){
20         scanf("%d", &arr[i]);
21     }
22     insertionSort(arr,n);

```

|   | Input               | Expected       | Got            |   |
|---|---------------------|----------------|----------------|---|
| ✓ | 5<br>30 40 50 20 10 | 10 20 30 40 50 | 10 20 30 40 50 | ✓ |

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

# CS23232-Fundamentals of Data Structures Using C-2025 Batch

|              |                                |
|--------------|--------------------------------|
| Started on   | Saturday, 16 May 2026, 9:46 PM |
| State        | Finished                       |
| Completed on | Saturday, 16 May 2026, 9:57 PM |
| Time taken   | 11 mins                        |
| Name         | BHUVANA S 2025-BME             |

## Question 1

Correct

Marked out of 100

Flag question

Write a program to sort a list of integers in **ascending order** using the **Selection Sort** algorithm.

Selection Sort works by repeatedly selecting the **smallest element** from the unsorted portion of the array and placing it at the correct position in the sorted portion.

### Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

### Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5  
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

| Input               | Result         |
|---------------------|----------------|
| 5<br>30 40 50 20 10 | 10 20 30 40 50 |

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main(){
3     int n,i,j,min_idx,temp;
4     if(scanf("%d", &n)!=1)
5         return 0;
6     int arr[n];
7     for(i=0;i<n;i++){
8         scanf("%d", &arr[i]);
9     }
10    for(i=0;i<n-1;i++){
11        min_idx=i;
12        for(j=i+1;j<n;j++){
13            if(arr[j]<arr[min_idx]){
14                min_idx=j;
15            }
16        }
17        temp=arr[min_idx];
18        arr[min_idx]=arr[i];
19        arr[i]=temp;
20    }
21    for(i=0;i<n;i++){
22        printf("%d %s",arr[i],(i==n-1)?"":" ");

```

|   | Input               | Expected       | Got            |   |
|---|---------------------|----------------|----------------|---|
| ✓ | 5<br>30 40 50 20 10 | 10 20 30 40 50 | 10 20 30 40 50 | ✓ |

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)

# CS23232-Fundamentals of Data Structures Using C-2025 Batch

|                     |                                 |
|---------------------|---------------------------------|
| <b>Started on</b>   | Saturday, 16 May 2026, 9:57 PM  |
| <b>State</b>        | Finished                        |
| <b>Completed on</b> | Saturday, 16 May 2026, 10:05 PM |
| <b>Time taken</b>   | 7 mins 55 secs                  |
| <b>Name</b>         | BHUVANA S 2025-BME              |

## Question 1

Correct

Marked out of 100

Flag question

Write a program to sort a list of integers in **ascending order** using the **Bubble Sort** algorithm.

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. This process continues until the array is fully sorted.

### Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

### Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5  
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

| Input               | Result         |
|---------------------|----------------|
| 5<br>30 40 50 20 10 | 10 20 30 40 50 |

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main(){
3     int n,i,j,temp;
4     if(scanf("%d", &n)!=1)
5         return 0;
6     int arr[n];
7     for(i=0;i<n;i++){
8         scanf("%d", &arr[i]);
9     }
10    for(i=0;i<n-1;i++){
11        for(j=0;j<n-i-1;j++){
12            if(arr[j]>arr[j+1]){
13                temp=arr[j];
14                arr[j]=arr[j+1];
15                arr[j+1]=temp;
16            }
17        }
18    }
19    for(i=0;i<n;i++){
20        printf("%d %s",arr[i],(i==n-1)?"":" ");
21    }
22    return 0;

```

|   | Input               | Expected       | Got            |   |
|---|---------------------|----------------|----------------|---|
| ✓ | 5<br>30 40 50 20 10 | 10 20 30 40 50 | 10 20 30 40 50 | ✓ |

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)

# CS23232-Fundamentals of Data Structures Using C-2025 Batch

|              |                                 |
|--------------|---------------------------------|
| Started on   | Saturday, 16 May 2026, 10:07 PM |
| State        | Finished                        |
| Completed on | Saturday, 16 May 2026, 10:23 PM |
| Time taken   | 15 mins 38 secs                 |
| Name         | BHUVANA S 2025-BME              |

**Question 1**

Correct

Marked out of 100

Flag question

Write a program to sort a list of integers in **ascending order** using the **Quick Sort** algorithm.

Quick Sort is a **divide and conquer** technique that works by selecting a **pivot element**, partitioning the array into two subarrays, and recursively sorting them.

**Input Format:**

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

**Output Format:**

- Print the sorted array in **ascending order**.

Sample Input:

5  
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

| Input               | Result         |
|---------------------|----------------|
| 5<br>30 40 50 20 10 | 10 20 30 40 50 |

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 void swap(int*a,int*b){
3     int t=*a;
4     *a=*b;
5     *b=t;
6 }
7 int partition(int arr[],int low,int high){
8     int pivot=arr[high];
9     int i=(low-1);
10    for(int j=low;j<=high-1;j++){
11        if(arr[j]<pivot){
12            i++;
13            swap(&arr[i],&arr[j]);
14        }
15    }
16    swap(&arr[i+1],&arr[high]);
17    return(i+1);
18 }
19 void quickSort(int arr[],int low,int high){
20    if(low<high){
21        int pi=partition(arr,low,high);
22        quickSort(arr,low,pi-1);
```

|   | Input               | Expected       | Got            |   |
|---|---------------------|----------------|----------------|---|
| ✓ | 5<br>30 40 50 20 10 | 10 20 30 40 50 | 10 20 30 40 50 | ✓ |

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)

# CS23232-Fundamentals of Data Structures Using C-2025 Batch

|                     |                                 |
|---------------------|---------------------------------|
| <b>Started on</b>   | Saturday, 16 May 2026, 10:23 PM |
| <b>State</b>        | Finished                        |
| <b>Completed on</b> | Saturday, 16 May 2026, 10:55 PM |
| <b>Time taken</b>   | 31 mins 32 secs                 |
| <b>Name</b>         | BHUVANA S 2025-BME              |

## Question 1

Correct

Marked out of 100

Flag question

Write a program to sort a list of integers in **ascending order** using the **Merge Sort** algorithm.

Merge Sort is a **divide and conquer** algorithm that divides the array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays to produce the final sorted array.

### Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

### Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5  
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

| Input               | Result         |
|---------------------|----------------|
| 5<br>30 40 50 20 10 | 10 20 30 40 50 |

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 void merge(int arr[],int l,int m,int r){
4     int i,j,k;
5     int n1=m-l+1;
6     int n2=r-m;
7     int L[n1],R[n2];
8     for(i=0;i<n1;i++) L[i]=arr[l+i];
9     for(j=0;j<n2;j++) R[j]=arr[m+1+j];
10    i=0;j=0;k=l;
11    while(i<n1 && j<n2){
12        if(L[i]<=R[j])arr[k++]=L[i++];
13        else arr[k++]=R[j++];
14    }
15    while(i<n1)arr[k++]=L[i++];
16    while(j<n2)arr[k++]=R[j++];
17 }
18 void mergeSort(int arr[],int l,int r){
19     if(l<r){
20         int m=l+(r-l)/2;
21         mergeSort(arr,l,m);
22         mergeSort(arr,m+1,r);

```

|   | Input               | Expected       | Got            |   |
|---|---------------------|----------------|----------------|---|
| ✓ | 5<br>30 40 50 20 10 | 10 20 30 40 50 | 10 20 30 40 50 | ✓ |

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)